JavaScript Fundamentals

Student Notes & Interview Preparation Guide

Topics covered: Outputs | Identifiers | Variables | Scope | Data Types | Operators | Type Conversion | Conditional Statements | Loops | Functions

1. JavaScript Outputs

Why Do We Need Outputs?

When we write a program, we need a way to display the result to the user or developer.

Different Ways to Display Output

1. console.log()

Displays output in the browser console.

```
console.log("Hello Everyone");
```

Use Case: Used mostly for debugging and testing.

2. alert()

Displays a popup message.

```
alert("Welcome");
```

3. document.write()

Writes content directly to the webpage.

```
document.write("Hello World");
```

4. innerHTML

Changes the content of an HTML element.

```
document.getElementById("hello").innerHTML = "Hello Everyone";
```

Q: Which output method is most commonly used by developers?
Answer: console.log() because it helps in debugging.

---

2. JavaScript Identifiers

What are Identifiers?

Identifiers are names given to variables, functions, classes, etc.

Rules

An identifier can contain:

- A-Z
- a-z
- $
- _

- Numbers (but not at the beginning)

Valid Examples

```
name
Name1
$user
_name
```

Invalid Example

```
4name
```

Cannot start with a number.

Q: Can an identifier start with a number?
Answer: No.

---

3. JavaScript Variables

Why Variables?

Variables store data that can be used later.

Types of Variables

1. var

```
var age = 25;
```

2. let

```
let city = "Hyderabad";
```

3. const

```
const pi = 3.14;
```

Re-declaration vs Re-assignment

Variable	Re-declaration	Re-assignment
var	Yes	Yes
let	No	Yes
const	No	No

Example

```
var name = "John";
```

```
var name = "David";
```

Valid because var allows redeclaration.

Scope of Variables

Global Scope

Accessible everywhere.

Block Scope

Accessible only inside { }.

Example using let

```
let num = 10;

{
    let num = 20;
}

console.log(num);
```

Output:

```
10
```

Example using var

```
var num = 10;

{
    var num = 20;
}

console.log(num);
```

Output:

```
20
```

Q: Which variables are block scoped?
Answer: let and const.

---

4. JavaScript Data Types

What is a Data Type?

A data type tells JavaScript what kind of value a variable stores.

Primitive Data Types

String

```
let name = "JavaScript";
```

Number

```
let age = 25;
```

BigInt

```
let bigNumber = 12345678901234567890n;
```

Boolean

```
let isActive = true;
```

Undefined

```
let x;
```

Null

```
let value = null;
```

Non-Primitive Data Types

Object

```
let person = {
  name: "John",
  age: 25
};
```

Array

```
let colors = ["red", "blue", "green"];
```

Function

```
function hello() {}
```

Date

```
let today = new Date();
```

typeof Operator

```
typeof "Hello"
```

Output:

```
string
```

Important Interview Question

```
typeof null
```

Output:

```
object
```

This is a well-known JavaScript bug.

Explain the difference between null and undefined

Intro Line:

“Null and undefined are both values in JavaScript used to represent the absence of a meaningful value, but they are used in different contexts and signify different things."

3 Points of Comparison:

Definition and Default Values:

- “Undefined in JavaScript is a type that represents a variable that has been declared but not assigned a value. It's the default value for variables and parameters. For instance, if you declare let a;, a is automatically assigned the value undefined.
- “Null, on the other hand, is an assignment value. It can be assigned to a variable as a representation of no value. Unlike undefined, null must be explicitly assigned. For example, let a = null; means you are intentionally assigning 'no value' to a.”

Type and Usage:

- “In terms of type, undefined is actually its own type (undefined), whereas null is considered an object in JavaScript. This is a bit of a quirk in JavaScript because null is not really an object but rather a primitive value.”
- “When it comes to usage, you typically see undefined when JavaScript itself doesn't know what a value is, like with uninitialized variables or missing function parameters. Null is used when you, as the programmer, want to intentionally denote that a variable should have 'no value' or represent an 'empty' or 'unknown' state.”

Equality:

- “When comparing null and undefined with the equality operator (==), they are equal to each other. However, using the strict equality operator (===), they are not equal because they are of different types.”
- "This distinction is important in code where you want to check if a variable either doesn't exist or is explicitly set to 'no value', or when you want to differentiate between the two states."
-

5. JavaScript Operators

What are Operators?

Operators perform operations on values.

Arithmetic Operators

```
+
-
*
/
%
**
```

Example

```
console.log(2 ** 5);
```

Output:

```
32
```

Assignment Operators

```
=
+=
-=
*=
/=
```

Example

```
let x = 10;

x += 5;
```

Output:

```
15
```

Comparison Operators

```
<
>
<=
>=
==
===
!=
!==
```

== vs ===

```
100 == "100"
```

Output:

```
true
```

Because type conversion occurs.

```
100 === "100"
```

Output:

```
false
```

Because data types are different.

Logical Operators

```
&&
||
!
```

Ternary Operator

Short form of if-else.

```
condition ? trueValue : falseValue;
```

Example

```
num === 1000 ? "Yes" : "No";
```

Q: Difference between == and ===?
Answer: == checks value only. === checks both value and datatype.

---

6. JavaScript Type Conversion

What is Type Conversion?

Converting one datatype into another datatype.

Implicit Conversion

Done automatically by JavaScript.

Example

```
51 + 81 + "JavaScript"
```

Output:

```
"132JavaScript"
```

```
"5" - 2
```

Output:

```
3
```

```
"5" + 2
```

Output:

```
"52"
```

Explicit Conversion

Done manually by the developer.

String Conversion

```
String(123)
```

Output:

```
"123"
```

Number Conversion

```
Number("123")
```

Output:

```
123
```

Boolean Conversion

```
Boolean(0)
```

Output:

```
false
```

```
Boolean("Hello")
```

Output:

```
true
```

Q: What is Type Coercion?
Answer: Automatic datatype conversion performed by JavaScript.

7. Conditional Statements

Why Conditions?

To make decisions in programs.

if Statement

```
if(age >= 18){
    console.log("Eligible");
}
```

if-else

```
if(age >= 18){
    console.log("Eligible");
}
else{
    console.log("Not Eligible");
}
```

if else if

```
if(score > 90){
    console.log("A Grade");
}
else if(score > 75){
    console.log("B Grade");
}
else{
    console.log("C Grade");
}
```

switch Statement

```
switch(day){
    case 1:
        console.log("Monday");
        break;

    default:
        console.log("Invalid Day");
}
```

Q: Why is break used in switch?
Answer: To stop execution of remaining cases.

8. JavaScript Loops

Why Loops?

To execute the same block of code multiple times.

for Loop

```
for(let i=0; i<5; i++){
    console.log(i);
}
```

while Loop

```
let i = 0;

while(i < 5){
    console.log(i);
    i++;
}
```

do-while Loop

```
let i = 0;

do{
    console.log(i);
    i++;
}
while(i < 5);
```

for...in Loop

Used for object keys and array indexes.

```
let person = {
    name: "John",
    age: 25
};

for(let key in person){
    console.log(key);
}
```

Output:

```
name
age
```

for...of Loop

Used for values.

```
let arr = ["a", "b", "c"];

for(let value of arr){
    console.log(value);
}
```

Output:

```
a
b
c
```

Interview Question: for...in vs for...of

for...in	for...of
Iterates over keys / indexes	Iterates over values
Used with objects	Used with arrays & strings

---

9. JavaScript Functions

What is a Function?

A reusable block of code that performs a specific task.

Function Syntax

```
function functionName(){
    // code
}
```

Non-Parameterized Function

```
function hello(){
    console.log("Hello");
}
```

Parameterized Function

```
function add(a,b){
    return a+b;
}
```

Function Call

```
add(5,10);
```

Return Keyword

```
function square(num){
    return num * num;
}
```

Types of Functions

1. Function Declaration

```
function add(a,b){
    return a+b;
}
```

2. Function Expression

```
const add = function(a,b){
    return a+b;
}
```

3. Arrow Function

```
const add = (a,b) => a+b;
```

Arrow Function with One Parameter

```
const print = value => console.log(value);
```

Interview Questions

Q: What is a function?
Answer: A reusable block of code that performs a specific task.

Q: What is the difference between Function Declaration and Function Expression?
Answer: Function declarations are named functions, while function expressions are stored inside variables.

Q: Why are Arrow Functions popular?
Answer: Because they provide shorter and cleaner syntax.

Practice Questions

1. Create variables using var, let, and const.
2. Create examples for all primitive data types.
3. Write a program using all arithmetic operators.
4. Create a calculator using if-else.
5. Print numbers 1 to 20 using loops.

6. Find the sum of the first 10 numbers using a loop.
7. Create a function to find the square of a number.
8. Convert a string into a number using explicit conversion.
9. Create an object and print all keys using for...in.
10. Create an array and print all values using for...of.

JavaScript Fundamentals Summary

- Outputs
- Identifiers
- Variables (var, let, const)
- Scope
- Data Types
- Operators
- Type Conversion
- Conditional Statements
- Loops
- Functions
- Function Declaration
- Function Expression
- Arrow Functions

These topics form the foundation required before moving to Arrays, Strings, Objects, DOM Manipulation, Events, ES6 Features, and React.